

原文ECMNet:Lightweight Semantic Segmentation with Efficient CNN-Mamba Network

1. Motivation

- a. 将 Mamba-inspired Selective Scan机制融入医学图像分割任务中，轻量的、Mamba-inspired 模块 —— **SS3D-CMF**处理：
 - i. 3D体数据(3D volume)
 - ii. 在多模态融合(multi-modal)-不同物质机理产生的图像数据中的全局感知建模能力，尤其关注在边缘设备部署、高效推理等临床实际需求下的可行性与实用性。

2. Proposed Framework

- a. SS3D-CMF 模块核心思想

- i. 输入:多模态 3D 图像(如 PET-CT, T1/T2/FLAIR MRI), 形状为:

$$x_A, x_B \in \mathbb{R}^{C \times D \times H \times W}$$

- ii. 目标:用模态**A**引导模态**B**的注意力建模, 实现跨模态的 **gated** 特征融合
 - iii. 操作流程:

$$G = \sigma(\text{Conv3D}(x_A)) \quad (\text{Gating from modality A})$$

$$z = \text{DWConv3D}(x_B), \quad y = \text{FFN}(z \odot G) + x_B$$

- iv. 这实际上是一种“模态 **A** 引导模态 **B** 的特征筛选”机制, 属于 gated cross-modal modulation, 等价于:在不同模态之间建立信息控制通道, 使得“信息重要性”由上下文模态自动决定, 而不是 naive 融合(如直接拼接或平均)。
 - v. 特征融合方式简洁、可并行, 完全卷积结构, 适用于现有 **TransUNet/UNETR/SwinUNet3D** 替换或插入。

```
class SS3D_CMF(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.dwconv = nn.Conv3d(in_channels, in_channels, 3, padding=1,
groups=in_channels)
        self.gate = nn.Sequential(
            nn.Conv3d(in_channels, in_channels, 1),
            nn.Sigmoid()
        )
        self.ffn = nn.Sequential(
            nn.Conv3d(in_channels, in_channels, 1),
            nn.ReLU(),
            nn.Conv3d(in_channels, in_channels, 1)
        )
```

```
def forward(self, x_target, x_guide):
    g = self.gate(x_guide)          # From guiding modality
    z = self.dwconv(x_target)       # From target modality
    out = self.ffn(z * g) + x_target # Residual connection
    return out
```

- b. SS3D-CMF 与 Mamba 的核心思想联系
- i. 本模块设计灵感源自 Mamba 中的状态空间建模与选择性扫描机制，其核心思想在 SS3D-CMF 中体现为：

Mamba 序列建模	SS3D-CMF 多模态建模	对应说明
输入	目标模态特征	当前待建模的特征输入
状态	引导模态特征	相当于前态或全局引导
状态更新	$z = \text{DWConv}(x_B), G = \sigma(\text{Conv3D}(x_A))$	引导模态控制目标模态的特征信息选择流动
Gate	选择性融合	动态门控机制控制特征保留与抑制
输出	输出	非线性变换后残差输出，保留语义并增强表达能力

- ii. 结论:SS3D-CMF 本质上是在空间维度使用一个“状态空间选择性建模机制”，其中 guide 模态(x_A)扮演 Mamba 的“状态 s_{t-1} ”角色，对 x_B 的特征流进行门控控制。
- c. 多模态扩展策略
- i. 多于两模态如何融合？

1. 多路交叉 Pairwise Gated 融合
- a. 任意模态对(如 T1-T2、T2-FLAIR、FLAIR-T1)两两交互：

```
z1 = SS3D_CMF(x_T1, x_T2)
z2 = SS3D_CMF(x_T2, x_FLAIR)
z3 = SS3D_CMF(x_FLAIR, x_T1)
y = Average([z1, z2, z3])
```

2. 主模态引导式融合
- a. 选择一个主模态(如 FLAIR)统一引导其他模态：

```
y1 = SS3D_CMF(x_T1, x_FLAIR)
y2 = SS3D_CMF(x_T2, x_FLAIR)
y = Average([y1, y2])
```

3. 这两种策略可泛化到 N 个模态, 既保留 gated 信息选择机制, 也确保工程实现稳定高效。

d. 常见多模态组合

多模态组合	描述	临床目的
CT + PET	CT: 高分辨结构; PET: 代谢活跃区域	肿瘤识别/治疗靶区规划
MRI T1 + T2 + FLAIR	提供组织/液体/病灶边界对比	脑肿瘤、脱髓鞘病、炎症分析
MRI + Ultrasound	高对比解剖结构 + 实时影像	神经介入导航、超声增强检测
多序列MRI	DWI, ADC, SWI, T1/T2*等	癫痫、脑卒中、功能分析

3. 实验配置

- a. 数据集BraTS(T1/T2/FLAIR), PET-CT, KiTS、AMOS
- b. 网络结构在 UNETR 或 SwinUNet3D 中引入 SS3D-CMF 替代 attention block
- c. 指标Dice, mIoU, Params, FLOPs, FPS
- d. Baseline UNet, TransUNet, UNETR, SwinUNet3D

4. 研究贡献总结

- a. 提出 SS3D-CMF: 首个用于 3D 多模态医学图像分割的选择性扫描建模模块, 结合跨模态 gated 注意力与空间建模能力;
- b. 设计完全基于 3D 卷积结构, 无需 Transformer, 自带位置感知, 计算线性可扩展
- c. 模块部署简单, 兼容各类主干网络, 可插拔式融合结构
- d. 在多模态 3D 医学任务中, 实现精度与效率的双提升

原文DA-TransUNet: integrating spatial and channel dual attention with transformer U-net for medical image segmentation

1. Motivation

a. DA-TransUNet 融合：

- i. 双注意力 DA-Block (PAM + CAM)
- ii. Transformer 全局建模

b. 但

- i. PAM 全局自注意力 $O(N^2)$ 计算开销大
- ii. CAM 全通道 $O(C^2)$ 耗资源
- iii. 固定通道压缩比不灵活
- iv. 不考虑硬件部署

c. 目标：

- i. 保留 DA-TransUNet 高精度，降低参数、计算量、内存，并适应不同硬件

2. Proposed Framework

a. PAM $\rightarrow$ Swin + Low-Rank Decomposition

- i. 现状：PAM 计算全图关系，复杂度 $O(N^2)$
- ii. 改进：

1. Swin Window Attention

- a. 将图像划分 $M \times M$ 小块，仅在局部 window 内做注意力，复杂度 $O(N \cdot M^2)$

2. Low-Rank 分解

- a. 将 window 内注意力矩阵 A 分解为：

$$A \approx PQ$$

$$P \in \mathbb{R}^{M^2 \times r}, Q \in \mathbb{R}^{r \times M^2}, r \ll M^2$$

- 复杂度 $O(2 \cdot M^2 \cdot r)$

3. 优点：

- a. 大幅减少 FLOPs
- b. 保留局部空间信息

b. CAM $\rightarrow$ Group Attention

- i. 现状：CAM 需计算 $C \times C$ 相似度矩阵，复杂度 $O(C^2)$
- ii. 改进：

1. 将通道分组：

- a. 每组单独做 attention
- b. 拼接回全局，复杂度降为 $O(C^2/G)$

iii. 优点：

1. 大幅降低参数
2. 保留通道相关性

c. Adaptive Channel Reduction

- i. 现状：DA-Block 全部采用固定 1/16 通道数

1. 改进:

a. Learnable ratio:

$$r = \sigma(W \cdot F)$$

2. 根据不同任务动态调整

a. 优点:

- i. 避免过度压缩
- ii. 对 small objects segmentation 特别有用

d. Hardware-Aware Configuration

i. 在推理时:

- 1. 检测 GPU memory
- 2. 动态决定:
 - a. Window size
 - b. Low-rank rank
 - c. Group size
 - d. Channel ratio

例如:

```
def hardware_config(free_mem):  
    if free_mem > 8:  
        return {"rank":64, "window":14, "groups":4}  
    elif free_mem > 4:  
        return {"rank":32, "window":7, "groups":8}  
    else:  
        return {"rank":16, "window":7, "groups":16}
```

3. Training Strategy

a. 分阶段训练:

- i. freeze encoder
- ii. train adaptive modules

b. Loss:

$$\text{TotalLoss} = \text{Dice} + \lambda \cdot \text{ComputePenalty}$$

4. Experiment Plan

a. Datasets:

- i. Synapse
- ii. CVC-ClinicDB Chest X-ray
- iii. ISIC2018

b. Experiments:

- i. AdaDA vs. static DA-TransUNet

- ii. Hardware-constrained tests
- c. Ablation:
 - i. Swin vs. global PAM
 - ii. Low-rank vs. full-rank
 - iii. Group vs. standard CAM
- d. Metrics:
 - i. Dice/IoU/Hausdorff Distance/Memory usage
- 5. Contributions
 - a. 将 Swin + Low-Rank 融入 PAM
 - b. 用 Group Attention 优化 CAM
 - c. 提出 Adaptive channel ratio
 - d. 提出 hardware-aware dynamic configuration
 - e. 保留 DA-TransUNet 精度, 大幅降低计算和内存
- 6. Challenges
 - a. Low-Rank rank selection
 - b. Group assignment training难度
 - c. CUDA kernel 是否支持动态维度
 - d. Runtime latency
- 7. Next Steps
 - a. integrate into DA-TransUNet
 - b. test on Synapse & ISIC2018
 - c. profile latency & memory

Title: AdaDA-TransUNet — Lightweight Dual-Attention Transformer UNet for Medical Image Segmentation

原文EMF-former: An Efficient and Memory-Friendly Transformer for Medical Image Segmentation

1. Motivation

- a. EMF-former 提出了:
 - i. DSPConv → 降低卷积参数
 - ii. VAA → 降低注意力复杂度
 - iii. S-MHA → 减少多头冗余和内存
- b. 局限:
 - i. 全部是静态设计
 - ii. 不适应不同病人、区域或硬件
 - iii. 不适合小目标 segmentation
- c. 提出 AdaEMF:
 - i. adaptive channel selection + adaptive attention pooling + learnable S-MHA gating + hardware-aware config

2. AdaEMF Framework

- a. Adaptive DSPConv Channel Selection
 - i. 现状: 固定前 1/4 channel 做 DWConv。问题: 不同 feature density 不同, 固定 1/4 未必最优
 - ii. Proposed: 学习 dynamic gating vector:
 - 1. 保留重要 spatial channels, 尤其对 small targets 有帮助。

$$g_c = \sigma(W \cdot F)$$

伪代码:

```
def adaptive_dspconv(features):
    global_feat = torch.mean(features, dim=[2,3])
    gates = torch.sigmoid(
        nn.Linear(global_feat_dim, features.shape[1])(global_feat)
    )
    output = []
    for c in range(features.shape[1]):
        if gates[0, c] > 0.5:
            out_c = depthwise_conv(features[:,c,:,:].unsqueeze(1))
        else:
            out_c = pointwise_conv(features[:,c,:,:].unsqueeze(1))
        output.append(out_c)
    return torch.cat(output, dim=1)
```

- b. Adaptive Vector Aggregation Attention (VAA)
 - i. 现状: mean pooling。问题: 容易被 background 主导。
 - ii. Proposed: weighted mean 或 hybrid pooling:

$$K_{\text{global}} = \sum_{i=1}^N w_i \cdot X_i$$

或：

$$K_{\text{global}} = \alpha \cdot \text{mean}(X) + (1 - \alpha) \cdot \text{median}(X)$$

伪代码：

```
def adaptive_vaa(features):
    weights = torch.softmax(
        nn.Linear(features.shape[1], 1)(features), dim=0
    )
    global_token = (features * weights).sum(dim=0)
    return global_token
```

c. Learnable Serial Multi-Head Attention (S-MHA)

- i. 现状：奇数 head 计算，偶数 head skip。
- ii. Proposed: learnable gating:

$$g_c = \sigma(W \cdot F)$$

伪代码：

python

```
def learnable_s_mha(global_feat, Q, K, V, num_heads):
    gates = torch.sigmoid(
        nn.Linear(global_feat.shape[1], num_heads)(global_feat)
    )
    outputs = []
    for i in range(num_heads):
        if gates[0, i] > 0.5:
            head = attention(Q[i], K[i], V[i])
        else:
            head = torch.zeros_like(Q[i])
        outputs.append(head)
    return sum(outputs)
```

d. Hardware-aware Runtime Configurator

- i. 检测 GPU memory, 动态调整 resolution、channel、head 数。

示例代码:

```
def hardware_aware_config(global_feat):
    free_mem = check_gpu_memory()
    if free_mem > 8:
        return {"res":1.0, "channels":128, "heads":8}
    elif free_mem > 4:
        return {"res":0.5, "channels":64, "heads":4}
    else:
        return {"res":0.5, "channels":32, "heads":2}
```

3. Training Strategy

- a. 两阶段训练:
 - i. freeze encoder
 - ii. train adaptive modules
- b. Loss:

$$\text{TotalLoss} = \text{Dice} + \lambda \times \text{ComputePenalty}$$

- c. Datasets:
 - i. ACDC
 - ii. Hippocampus
 - iii. BTCV
- d. Experiments:
 - i. Adaptive vs. static EMF-former
 - ii. Hardware-constrained test
 - iii. Small target segmentation
 - iv. FLOPs & latency comparison

4. Expected Contributions

- a. 首次将 adaptive decoding 引入 EMF-former
- b. 保留 lightweight 同时智能分配资源
- c. 实现 hardware-aware deployment
- d. 对 small object segmentation 改善显著

5. Potential Challenges

- a. gating module 训练难度
- b. CUDA kernel 支持 dynamic shape
- c. runtime switching latency

6. Next Steps

- a. implement prototype on EMF-former baseline
- b. test on ACDC + Hippocampus
- c. collect memory/latency data

Possible Title: AdaEMF: Hardware-Aware Adaptive Memory-Friendly Transformer for Medical Image Segmentation

原文EffiDec3D: An Optimized Decoder for High-Performance and Efficient 3D Medical Image Segmentation

1. Motivation

- a. EffiDec3D 显著优化了 3D segmentation decoder:
 - i. 通过全局 channel reduction
 - ii. 使用 lower-resolution 输出
- b. 然而仍有局限:
 - i. 是静态架构
 - ii. 不考虑:
 - 1. 每张影像难度差异
 - 2. 病人解剖学差异
 - 3. 硬件环境差异
 - iii. 因此提出 AdaDec3D:
 - 1. 动态适配:
 - a. resolution
 - b. channel width
 - 2. 同时 hardware-aware
 - a. 目标:
 - i. 提高效率
 - ii. 保持精度
 - iii. 可部署于不同硬件环境

2. AdaDec3D Framework

- a. AdaDec3D 主要包含三个核心模块:
 - i. Adaptive Resolution Selection
 - 1. EffiDec3D:
 - a. 全局使用 full-res 或 half-res
 - 2. AdaDec3D:
 - a. 每张图像根据 encoder global features, 预测最佳 resolution 配置:
 - i. 全局 1×
 - ii. 全局 1/2×
 - iii. hybrid(ROI high-res, 背景 low-res)

```
def predict_resolution(global_feat):  
    # 简单 FC + sigmoid  
    res_prob = torch.sigmoid(  
        nn.Linear(global_feat_dim, 1)(global_feat)  
    )  
    if res_prob > 0.5:  
        return 1.0 # full resolution  
    else:  
        return 0.5 # half resolution
```

3. Adaptive Channel Reduction

- a. EffiDec3D:
 - i. 全局使用 encoder skip connection 中最小 channel 数
- b. AdaDec3D:
 - i. 对每个 skip connection: 动态预测最优 channel 数

```
def predict_channels(global_feat, max_channels):
    # 将 global feature map 到 gating vector
    gates = torch.sigmoid(
        nn.Linear(global_feat_dim, max_channels)(global_feat)
    )
    effective_channels = int(gates.sum().item())
    return effective_channels
```

4. Hardware-aware Configurator

- a. EffiDec3D:
 - i. 不考虑硬件约束
- b. AdaDec3D:
 - i. 在推理阶段:
 - 1. 检测 GPU memory
 - 2. 动态调整:
 - a. resolution
 - b. channel width
 - c. decoder depth

```
import torch

def check_memory():
    return torch.cuda.mem_get_info()[0] / (1024**3) # GB

def adjust_config(mem_free):
    if mem_free > 8:
        return {"res":1.0, "channels":128}
    elif mem_free > 4:
        return {"res":0.5, "channels":64}
    else:
        return {"res":0.5, "channels":32}
```

3. Integration with EffiDec3D

- a. EffiDec3D GitHub: <https://github.com/SLDGroup/EffiDec3D>
- b. 插入位置:
 - i. ChannelReductionResidualBlock: 集成 gating logic
 - ii. Decoder: 集成 adaptive resolution

Example: dynamic channels

原始 EffiDec3D:

```
self.conv1 = nn.Conv3d(in_channels, out_channels, 3, padding=1)
```

AdaDec3D:

```
eff_channels = predict_channels(global_feat, out_channels)
self.conv1 = nn.Conv3d(in_channels, eff_channels, 3, padding=1)
```

4. Training Strategy

- a. 建议分两阶段:
 - i. 先训练 baseline EffiDec3D, freeze encoder
 - ii. 再训练 adaptive 模块:
 1. resolution predictor
 2. channel predictor
 3. memory-aware thresholds
- b. Loss:
 - i. dice loss
 - ii. 可加 resource penalty:

$$\text{TotalLoss} = \text{DiceLoss} + \lambda \cdot \text{MemoryPenalty}$$

5. Experiment Plan

- a. Datasets: FeTA/BTCV/MSD
- b. Metrics:
 - i. Dice
 - ii. memory usage
 - iii. Latency
- c. Albation:
 - i. Adaptive vs. static config
 - ii. Hardware-constrained tests
 - iii. 小目标 voxel-level precision

6. Expected Contributions

- a. 提出 first hardware-aware adaptive decoder
- b. Dynamic allocation:
 - i. Resolution
 - ii. Channels
- c. 实现 truly deployable 3D segmentation
 - i. 在保持精度下, 显著降低 FLOPs 和 memory

7. Potential Challenges

- a. Gating function 如何训练稳定?
- b. CUDA kernel 是否支持 dynamic channels?
- c. runtime switching 需要非常快

8. Next Steps

- a. 在 EffiDec3D 代码上 prototype
- b. 在 FeTA 和 BTCV 上测试

c. 分析 memory/latency scaling

References

- EffiDec3D: <https://github.com/SLDGroup/EffiDec3D>

Title: AdaDec3D: Hardware-Aware Adaptive Decoder for 3D Medical Image Segmentation

原文G-CASCADE: Efficient Cascaded Graph Convolutional Decoding for 2D Medical Image Segmentation

1. 研究动机与目标

a. 现状问题:

- i. G-CASCADE 使用基于 KNN 的 GCN 构图方式, 结构静态、非方向性、不感知语义或空间布局, 难以处理如多器官、对称结构或高分辨图中的长距离依赖。
- ii. 解码器中的 SPA 使用 7×7 卷积进行空间注意力建模, 对高分辨图(如 512×512 CT/MRI)仍存在计算瓶颈。
- iii. 在小样本、低对比度、对称结构等医疗图像数据集上, 结构信息(如位置、形状)感知能力更关键。

b. 研究目标:

- i. 提出一种 **Spatially-Aware Graph Decoding** 解码器框架, 集成:
 1. 局部注意力图(Locally Attentive Graph)作为动态可学习 **GCN** 图构建策略
 2. Coordinate Attention(CA)替代 SPA, 强化空间位置建模且降低计算成
- ii. 在多个具有挑战性的医学图像分割任务上验证其优越性(高分辨率、小样本、低对比度等)

2. 方法概述

a. G-CASCADE 解码器中的两个关键模块:

原始模块

替换/增强方向

GCB 图卷积中的 **KNN**图 用 Locally Attentive Graph (LAG) 动态建图(支持方向/可学习边)

SPA 空间注意力模块 用 Coordinate Attention (CA) 替换, 提升位置感知 + 降低 FLOPs

整体结构如下:

```
Encoder (PVT/MERIT)
  ↓
  X1, X2, X3, X4
  ↓
  GCAM (GCB with LAG) + CA (replace SPA)
  ↓
  UCB → SegHead
  ↓
  Final segmentation map
```

3. 模块数学细节

a. Locally Attentive Graph (LAG) 构图 (替代 KNN)

i. 我们将原始 KNN 图中边的定义从:

$$e_{ij} = \mathbb{I}(j \in \text{TopK}_{L_2}(i))$$

替换为基于 attention 的加权边:

$$e_{ij} = \frac{\exp(\phi(x_i, x_j))}{\sum_{k \in \mathcal{N}(i)} \exp(\phi(x_i, x_k))}$$

其中, $\phi(x_i, x_j)$ 是可学习的相似性函数, 例如 dot product 或 MLP:

$$\phi(x_i, x_j) = \text{MLP}([x_i || x_j || x_i - x_j])$$

然后图卷积操作为:

$$h'_i = \sum_{j \in \mathcal{N}(i)} e_{ij} \cdot W x_j$$

优点: 边是可学习的, 能适应结构变化; 加入空间偏置也可增强位置感知

ii. **Update**: 已有类似文章 UnSegMedGAT 用的点积注意力, 还可以考虑:

1. 余弦相似度 - 点积归一化后
2. MLP
3. 高斯核

b. Coordinate Attention (CA) 替换 SPA

i. SPA 的原始形式为:

$$\text{SPA}(x) = \sigma(\text{Conv}_{7 \times 7}([C_{\text{avg}}(x), C_{\text{max}}(x)])) \odot x$$

ii. CA 结构替换为如下公式:

1. 特征图

$$x \in \mathbb{R}^{C \times H \times W}$$

2. 沿 H/W 做平均池化:

$$z^h(c, h) = \frac{1}{W} \sum_{w=1}^W x(c, h, w), \quad z^w(c, w) = \frac{1}{H} \sum_{h=1}^H x(c, h, w)$$

3. 拼接后过共享 1x1 conv, 再分别输出两个注意力图, 最终输出为:

$$\text{CA}(x) = x \cdot a^h \cdot a^w$$

4. 实验设计

a. 数据集选择:

场景	数据集	特点
高分辨率	Retina Vessel Segmentation	血管细、图大
小样本	ACDC (心脏MRI)	仅100例
低对比	ISIC2018	皮肤病变边界模糊

多器官	Synapse CT	空间结构复杂
对称结构	CT肾脏/肺部(自建或公开)	左右对称测试建边合理性

b. 评价指标:

- i. 精度指标: DICE、IoU、HD95(边界感知)
- ii. 效率指标: FLOPs、Params、推理时延(ms)
- iii. 构图质量指标(如有): 邻接图熵、多样性、稳定性

5. 模块伪代码(PyTorch风格)

a. LAG-based GCB 构图伪代码(替换原始KNN)

```
class LAGraphConv(nn.Module):
    def __init__(self, in_channels, k=8):
        super().__init__()
        self.k = k
        self.sim_fn = nn.Sequential(
            nn.Linear(2 * in_channels, in_channels),
            nn.ReLU(),
            nn.Linear(in_channels, 1)
        )

    def forward(self, x): # x: [B, C, H, W]
        B, C, H, W = x.shape
        patches = x.flatten(2).transpose(1, 2) # [B, N, C]
        N = patches.size(1)

        # Compute dynamic graph edge weights
        i_feat = patches.unsqueeze(2).expand(-1, -1, N, -1)
        j_feat = patches.unsqueeze(1).expand(-1, N, -1, -1)
        edge_input = torch.cat([i_feat, j_feat], dim=-1) # [B, N, N, 2C]

        edge_score = self.sim_fn(edge_input).squeeze(-1) # [B, N, N]
        edge_weight = F.softmax(edge_score, dim=-1)

        out = torch.bmm(edge_weight, patches) # [B, N, C]
        out = out.transpose(1, 2).reshape(B, C, H, W)
        return out
```

b. Coordinate Attention 伪代码

```
class CoordinateAttention(nn.Module):
```



```

def __init__(self, channels, reduction=32):
    super().__init__()
    mid_channels = max(8, channels // reduction)
    self.conv1 = nn.Conv2d(channels, mid_channels, kernel_size=1)
    self.bn1 = nn.BatchNorm2d(mid_channels)
    self.act = nn.ReLU()
    self.conv_h = nn.Conv2d(mid_channels, channels, kernel_size=1)
    self.conv_w = nn.Conv2d(mid_channels, channels, kernel_size=1)

def forward(self, x):
    B, C, H, W = x.size()
    h_pool = F.adaptive_avg_pool2d(x, (H, 1)) # [B, C, H, 1]
    w_pool = F.adaptive_avg_pool2d(x, (1, W)) # [B, C, 1, W]

    y = torch.cat([h_pool, w_pool], dim=2) # [B, C, H+1, W] (pseudo)
    y = self.conv1(y)
    y = self.bn1(y)
    y = self.act(y)

    h_attn = self.conv_h(y[:, :, :H, :]) # [B, C, H, 1]
    w_attn = self.conv_w(y[:, :, H:, :]) # [B, C, 1, W]

    return x * torch.sigmoid(h_attn) * torch.sigmoid(w_attn)

```

Title: Efficient Spatially-Aware Graph Decoding for High-Resolution Medical Image Segmentation

原文SegFormer3D: an Efficient Transformer for 3D Medical Image Segmentation

1. Problem

- a. 现有 Hierarchical Transformer:
 - i. stride 固定
 - ii. compute浪费在大背景区域
 - iii. 无法灵活分配算力给 lesion 区域

2. Hypothesis

- a. 通过 block-level dynamic stride: 能在不损失 **segmentation** 精度下, 大幅降低计算和显存。

3. Key Innovations

- a. Predict block-wise stride map
- b. Region-wise block conv → 保留 local context
- c. Hardware-aware:
 - i. use cuDNN fixed stride conv
 - ii. avoid custom CUDA kernel
 - iii. easy PyTorch integration

4. Method

- a. **Importance Map**
 - i. 用 CNN 预测 stride map
 - ii. $\text{stride} \in \{1, 2, 4\}$
- b. **Block Division**
 - i. 根据 stride map 分 block
- c. **Region-wise Conv**
 - i. 对每 block crop
 - ii. 用 cuDNN 跑 fixed stride conv
 - iii. **Upsampling + Merge**
 - 1. 所有 feature map upsample
 - 2. 按 mask 拼回 full map

5. Challenges

- a. Region boundary leakage
- b. Block stitching overhead
- c. Dynamic memory management
- d. CUDA warp divergence (避免了)

6. Expected Benefits

- a. Compute ↓
- b. Memory ↓
- c. Segmentation精度 $\approx$ fixed stride
- d. 特别 benefit small lesions

7. Experiments

- a. Dataset:
 - i. Synapse CT
 - ii. ACDC

- iii. Hippocampus MRI
- b. Metrics:
 - i. Dice
 - ii. Hausdorff
 - iii. GPU memory
 - iv. Runtime
- c. Ablation:
 - i. block size
 - ii. stride choices
 - iii. dynamic vs. fixed stride
- 8. Novelty
 - a. Hierarchical transformer block-wise dynamic stride → 尚无人做
 - b. Practical hardware-aware design
 - c. Fits medical imaging unique needs
- 9. Engineering Plan
 - a. 不写自定义 CUDA kernel
 - b. 写 C++ wrapper 调 cuDNN conv
 - c. 封装为 PyTorch extension:

```
out = dynamic_stride_conv(input, stride_map, weight, ...)
```

一句话总结

Block-level dynamic stride convolution is the missing piece to make hierarchical transformers compute-efficient for medical image segmentation, with realistic engineering feasibility.

Title: Block-wise Dynamic Stride Convolution for Hierarchical Transformers in Medical Image Segmentation